

Natural Language Processing (Character Generation) with Recurrent Neural Network

We will generate character using a RNN. We will work with a dataset of Shakespeare's writing. Given a sequence of characters from this data ("Shakespear"), train a model to predict the next character in the sequence ("e"). Longer sequences of text can be generated by calling the model repeatedly.

- The model is character-based.
- The model is trained on small batches of text (100 characters each), and is able to generate a longer sequence of text with coherent structure.

This guide is based on the following: https://www.tensorflow.org/tutorials/text/text_generation

```
# %tensorflow_version 2.x
from keras.preprocessing import sequence
import keras
import tensorflow as tf
import os
import numpy as np
```

Download the Shakespeare Dataset

Here, we use an extract from a Shakespeare play for training. We can use our own text paragraph data and use it for network training.

```
path_to_file = tf.keras.utils.get_file('shakespeare.txt',
    'https://storage.googleapis.com/download.tensorflow.org/data/shakespeare.txt')

# If you load your own data file, use the following lines:
# from google.colab import files
# path_to_file = list(files.upload().keys())[0]
```

Read Contents of File

Let's look at the contents of the file.

```
# Read, then decode for py2 compat.
text = open(path_to_file, 'rb').read().decode(encoding='utf-8')
# length of text is the number of characters in it
print('Length of text: {} characters'.format(len(text)))

Length of text: 1115394 characters
```

```
# Take a look at the first 250 characters in text
print(text[:250])
```

First Citizen:

Before we proceed any further, hear me speak.

All:

Speak, speak.

First Citizen:

You are all resolved rather to die than to famish?

All:

Resolved. resolved.

First Citizen:

First, you know Caius Marcius is chief enemy to the people.

```
# The unique characters in the file
vocab = sorted(set(text))
print(f'{len(vocab)} unique characters')
```

65 unique characters

Encode

Encode each unique character as a different integer.

```
# Creating a mapping from unique characters to indices
char2idx = {u:i for i, u in enumerate(vocab)}
idx2char = np.array(vocab)

def text_to_int(text):
    return np.array([char2idx[c] for c in text])

text_as_int = text_to_int(text)

# Check how a part of the text is encoded
print("Text:", text[:13])
print("Encoded:", text_to_int(text[:13]))
```

Text: First Citizen

Encoded: [18 47 56 57 58 1 15 47 58 47 64 43 52]

Make a function that can convert the numeric values back to text.

```
def int_to_text(ints):
    try:
        ints = ints.numpy()
```

```

except:
    pass
return ''.join(idx2char[ints])

print(int_to_text(text_as_int[:13]))
First Citizen

```

Create Training Examples

Given a character, or a sequence of characters, what is the most probable next character? This is the task we are training the model to perform.

Our task is to feed the model a sequence and have it return the next character. This means we need to split our text data from above into many shorter sequences that we can pass to the model as training examples.

The training examples we prepare will use a *seq_length* sequence as input and a *seq_length* sequence as the output where that sequence is the original sequence shifted one letter to the right. For example:

input: Hell | output: ello

```

# Create a stream of characters from our text data
char_dataset = tf.data.Dataset.from_tensor_slices(text_as_int)

# Use the batch method to turn this stream of characters into batches
of desired length
seq_length = 100 # length of sequence for a training example
sequences = char_dataset.batch(seq_length+1, drop_remainder=True)
examples_per_epoch = len(text)//(seq_length+1)

# Create training examples / targets
# Use these sequences of length 101 and split them into input and
output.
def split_input_target(chunk): # for the example: hello
    input_text = chunk[:-1] # hell
    target_text = chunk[1:] # ello
    return input_text, target_text # hell, ello

dataset = sequences.map(split_input_target) # we use map to apply the
above function to every entry

for x, y in dataset.take(3):
    print("\n\nEXAMPLE\n")
    print("INPUT")
    print(int_to_text(x))
    print("\nOUTPUT")
    print(int_to_text(y))

```

EXAMPLE

INPUT

First Citizen:

Before we proceed any further, hear me speak.

All:

Speak, speak.

First Citizen:

You

OUTPUT

First Citizen:

Before we proceed any further, hear me speak.

All:

Speak, speak.

First Citizen:

You

EXAMPLE

INPUT

are all resolved rather to die than to famish?

All:

Resolved. resolved.

First Citizen:

First, you

OUTPUT

are all resolved rather to die than to famish?

All:

Resolved. resolved.

First Citizen:

First, you k

EXAMPLE

INPUT

now Caius Marcius is chief enemy to the people.

All:
We know't, we know't.

First Citizen:
Let us k

OUTPUT
ow Caius Marcius is chief enemy to the people.

All:
We know't, we know't.

First Citizen:
Let us ki

Make training batches.

```
BATCH_SIZE = 64
VOCAB_SIZE = len(vocab) # vocab is number of unique characters
EMBEDDING_DIM = 256
RNN_UNITS = 1024

# Buffer size to shuffle the dataset
# (TF data is designed to work with possibly infinite sequences,
# so it doesn't attempt to shuffle the entire sequence in memory.
# Instead,
# it maintains a buffer in which it shuffles elements).
BUFFER_SIZE = 10000

data = dataset.shuffle(BUFFER_SIZE).batch(BATCH_SIZE,
drop_remainder=True)
```

Build the Model

We use an embedding layer, a LSTM layer, and one dense layer that contains a node for each unique character in our training data. The dense layer will give us a probability distribution over all nodes.

```
def build_model(vocab_size, embedding_dim, rnn_units, batch_size):
    model = tf.keras.Sequential([
        # FIX: Add a dedicated Input layer to define the batch shape
        tf.keras.Input(batch_shape=[batch_size, None]),

        # Remove the batch_input_shape argument from the Embedding layer
        tf.keras.layers.Embedding(vocab_size, embedding_dim),

        tf.keras.layers.GRU(rnn_units,
                            return_sequences=True,
                            stateful=True,
```

```

        recurrent_initializer='glorot_uniform'),

        tf.keras.layers.Dense(vocab_size)
    ])
    return model

model = build_model(VOCAB_SIZE, EMBEDDING_DIM, RNN_UNITS, BATCH_SIZE)
model.summary()

```

Model: "sequential"

Layer (type) Param #	Output Shape	
embedding (Embedding) 16,640	(64, None, 256)	
gru (GRU) 3,938,304	(64, None, 1024)	
dense (Dense) 66,625	(64, None, 65)	

Total params: 4,021,569 (15.34 MB)

Trainable params: 4,021,569 (15.34 MB)

Non-trainable params: 0 (0.00 B)

Create a Loss Function

Our model will output a (64, sequence_length, 65) shaped tensor that represents the probability distribution of each character at each timestep for every sequence in the batch.

We will create the loss function.

Before we do that, let's have a look at a sample input and the output from the untrained model.

```

for input_example_batch, target_example_batch in data.take(1):
    example_batch_predictions = model(input_example_batch) # ask our
    # model for a prediction on our first batch of training data (64
    # entries)
    print(example_batch_predictions.shape, "# (batch_size,
    sequence_length, vocab_size)") # print out the output shape

```

```

(64, 100, 65) # (batch_size, sequence_length, vocab_size)

# we can see that the prediction is an array of 64 arrays, one for
# each entry in the batch
print(len(example_batch_predictions))
print(example_batch_predictions)

64
tf.Tensor(
[[[-4.11155168e-03  3.81348515e-03 -1.36489887e-02 ... -6.57931762e-03
   5.50510688e-03  7.16807134e-03]
 [ 6.56564627e-03  6.26417482e-03 -1.64002050e-02 ... -3.26909823e-03
   1.44300694e-02 -2.45400937e-03]
 [-6.68726582e-03 -6.68988330e-03 -1.97637156e-02 ... -2.58455053e-04
   1.54653927e-02 -6.35589333e-03]
 ...
 [ 9.01444443e-03 -1.17472662e-02  5.75046288e-03 ...  1.57076772e-03
  -3.32343625e-05  5.27139567e-03]
 [ 3.98157956e-03 -1.43662049e-02 -1.44160865e-02 ...  1.02158431e-02
  -1.05089997e-03 -6.22462435e-03]
 [ 2.91643734e-03  1.71035472e-02 -1.13299573e-02 ... -2.86779087e-03
  -7.73682725e-03 -3.43866786e-03]]

[[[-6.60803495e-03 -1.32498452e-02  4.46170056e-03 ... -4.99134883e-03
   1.68749853e-03  5.81500074e-03]
 [-1.25433989e-02 -5.14452532e-03 -3.04006087e-03 ... -8.61136522e-03
   3.66450753e-04  9.06767882e-03]
 [-1.12194754e-02  1.91875745e-03 -1.34496242e-02 ... -1.06738638e-02
   5.55745512e-03  1.28731634e-02]
 ...
 [-7.82948919e-04 -4.50727949e-03 -1.78308580e-02 ... -3.09676398e-03
   1.40474224e-02 -1.07310032e-02]
 [-3.27373063e-03  5.30245248e-03 -2.01741327e-02 ... -6.20703958e-03
   1.13168908e-02  2.26490479e-03]
 [-7.44071975e-03  1.65947732e-02  8.98668543e-03 ... -4.69669886e-03
  -1.08502638e-02  4.15727962e-03]]

[[ 5.45388320e-04 -9.32953507e-03 -4.53003403e-03 ... -3.64812044e-03
   6.54663425e-03 -6.05342584e-03]
 [ 1.22386720e-02 -1.54834986e-02 -1.11594191e-02 ... -1.94703341e-02
   9.34223086e-03 -2.25221831e-02]
 [ 7.90577754e-03 -1.84711758e-02 -2.40831338e-02 ... -5.54075837e-03
   6.83393795e-03 -2.15072520e-02]
 ...
 [ 1.50467576e-02  1.63358301e-02 -1.56579688e-02 ... -1.58799672e-03
  -3.26393545e-03  2.21899548e-03]
 [-9.72696580e-03  5.43046650e-03 -2.34616455e-03 ... -4.45201527e-04
  -3.79412388e-03  2.47448124e-03]
 [ 5.98700438e-03 -3.47794546e-03  7.42016593e-04 ...  1.16682220e-02
  -2.99172523e-03  1.61384209e-03]]

```

```

...
[[-4.11155168e-03  3.81348515e-03 -1.36489887e-02 ... -6.57931762e-03
  5.50510688e-03  7.16807134e-03]
 [-1.14367045e-02  5.44416159e-03 -1.30630694e-02 ... -7.65092066e-03
  4.90335235e-03  8.77766311e-03]
 [ 2.43836292e-03  7.52782449e-03 -1.51827978e-02 ... -2.16840720e-03
  1.31252501e-02 -1.61450636e-03]
...
[ 5.41074388e-03 -1.01965070e-02 -2.21140515e-02 ...  3.07855546e-03
  3.25175771e-03 -8.18260387e-03]
 [ 9.18555260e-03 -2.00256407e-02 -7.11624976e-04 ...  9.54540167e-03
 -6.64565153e-03  9.13838856e-04]
 [ 4.53169458e-04 -5.84615208e-03 -1.49294995e-02 ... -1.63959013e-03
  1.42959552e-03  8.90830718e-03]]

[[-1.40023464e-03 -9.58861783e-03 -1.67537611e-02 ...  8.10351688e-03
 -6.86084386e-04 -1.00510335e-02]
 [-6.80961693e-03 -9.64001869e-04 -2.35254820e-02 ... -3.98563221e-03
  7.22843502e-03  2.05230992e-03]
 [ 6.23659790e-03 -1.01543777e-02 -1.76318958e-02 ... -1.96215846e-02
  1.28767118e-02 -1.75128281e-02]
...
[-6.66367589e-03  5.43942442e-04 -1.84776001e-02 ... -1.28369797e-02
  1.17017869e-02  9.33257956e-03]
 [-6.60695322e-03 -1.43408170e-03 -1.40593890e-02 ... -1.05862143e-02
  1.11610685e-02  1.79646835e-02]
 [-5.41764311e-03 -8.56594928e-03 -2.39664204e-02 ...  2.96791899e-03
  6.73735328e-03 -1.88342063e-03]]

[[-9.11805499e-03  2.13459367e-03 -6.35099737e-03 ... -4.30527283e-03
 -2.80550215e-04  4.62542661e-03]
 [ 3.46759055e-03  5.25055034e-03 -1.20361708e-02 ... -5.85735892e-04
  9.40956175e-03 -3.66491731e-03]
 [ 1.16875637e-02  1.27384514e-02 -4.48154658e-03 ...  8.98926519e-03
  3.43830092e-04  5.70080243e-03]
...
[-1.06532723e-02 -1.19120590e-02 -7.78784789e-03 ... -7.00849760e-03
  9.07495897e-03  7.80255813e-03]
 [-1.50188198e-02 -3.89101729e-03 -9.17150639e-03 ... -1.02107488e-02
  5.85498009e-03  1.03034386e-02]
 [-9.80890542e-03 -1.06205661e-02 -1.96132995e-02 ...  2.85353558e-03
  3.00987391e-03 -3.55074438e-03]]], shape=(64, 100, 65),
dtype=float32)

# Let's examine one prediction
pred = example_batch_predictions[0]
print(len(pred))
print(pred)

```

notice this is a 2d array of length 100, where each interior array is the prediction for the next character at each time step

```
100
tf.Tensor(
[[-4.11155168e-03  3.81348515e-03 -1.36489887e-02 ... -6.57931762e-03
  5.50510688e-03  7.16807134e-03]
 [ 6.56564627e-03  6.26417482e-03 -1.64002050e-02 ... -3.26909823e-03
  1.44300694e-02 -2.45400937e-03]
 [-6.68726582e-03 -6.68988330e-03 -1.97637156e-02 ... -2.58455053e-04
  1.54653927e-02 -6.35589333e-03]
 ...
 [ 9.01444443e-03 -1.17472662e-02  5.75046288e-03 ...  1.57076772e-03
 -3.32343625e-05  5.27139567e-03]
 [ 3.98157956e-03 -1.43662049e-02 -1.44160865e-02 ...  1.02158431e-02
 -1.05089997e-03 -6.22462435e-03]
 [ 2.91643734e-03  1.71035472e-02 -1.13299573e-02 ... -2.86779087e-03
 -7.73682725e-03 -3.43866786e-03]], shape=(100, 65), dtype=float32)
```

and finally we look at a prediction at the first timestep

```
time_pred = pred[0]
print(len(time_pred))
print(time_pred)
# and its 65 values representing the probability of each character
occurring next
```

```
65
tf.Tensor(
[-4.11155168e-03  3.81348515e-03 -1.36489887e-02 -6.20029960e-03
  3.43687553e-03 -5.27491001e-03  1.80207007e-03 -2.34272936e-03
  1.42262727e-02  3.09404451e-03 -3.61143961e-04 -5.82646439e-03
  3.18151852e-03  5.26046939e-03  7.14940019e-03  5.24111930e-03
 -3.18592647e-03 -5.84024005e-03  2.02227407e-03 -1.05672749e-02
 -7.79821817e-03  9.78319347e-03 -4.56746249e-03  1.49373780e-03
 -7.76215596e-03 -1.21212415e-02 -3.31334420e-04 -8.62535555e-03
 -4.35164478e-03 -1.08148195e-02 -2.31362693e-03  2.31265910e-02
 -8.03915691e-03  3.19734472e-03 -1.41402455e-02  1.17359031e-03
  4.91190888e-03 -1.45485206e-03  4.34450852e-03 -3.31310881e-03
  9.44924541e-05  3.26442625e-03  7.21123070e-03 -1.49007645e-02
  3.36124375e-03  6.72703795e-03 -4.60073818e-03 -3.46800219e-03
 -2.60936748e-03  6.17200334e-04  9.84332245e-03  1.24786573e-03
  3.48628638e-03 -2.79432489e-03 -8.18803674e-04 -6.14955463e-03
 -6.22604787e-03 -4.07539215e-03  1.11400755e-03 -4.96559823e-03
 -9.22498293e-03 -3.67050967e-03 -6.57931762e-03  5.50510688e-03
  7.16807134e-03], shape=(65,), dtype=float32)
```

If we want to determine the predicted character we need to sample the output distribution (pick a value based on probability)

```
sampled_indices = tf.random.categorical(pred, num_samples=1)
```

```
# now we can reshape that array and convert all the integers to
numbers to see the actual characters
sampled_indices = np.reshape(sampled_indices, (1, -1))[0]
predicted_chars = int_to_text(sampled_indices)

predicted_chars # and this is what the model predicted for training
sequence 1

"fFq?pKlM&aqun'nvWcvwV,xw'Hbs-&qwlvKu.G'T&MigPfVgK?SXAfHN;ZGzakc?aPFA
srV? ZJ?JgVewHoPJ&R IFmqV'ClKZl"
```

So, we need to create a loss function that can compare that output to the expected output and give us some numeric value representing how close the two were.

```
def loss(labels, logits):
    return tf.keras.losses.sparse_categorical_crossentropy(labels,
logits, from_logits=True)
```

Compile the Model

We can think of our problem as a classification problem where the model predicts the probability of each unique letter coming next.

```
model.compile(optimizer='adam', loss=loss)
```

Create Checkpoints

Now we are going to setup and configure our model to save checkpoints as it trains. This will allow us to load our model from a checkpoint and continue training it.

```
# Directory where the checkpoints will be saved
checkpoint_dir = './training_checkpoints'

# FIX: Add the required .weights.h5 extension to the filename
checkpoint_prefix = os.path.join(checkpoint_dir,
"ckpt_{epoch}.weights.h5")

checkpoint_callback = tf.keras.callbacks.ModelCheckpoint(
    filepath=checkpoint_prefix,
    save_weights_only=True)
```

Train the Model

If this is taking a while go to Runtime > Change Runtime Type and choose "GPU" under hardware accelerator.

```
history = model.fit(data, epochs=30, callbacks=[checkpoint_callback])
```

Epoch 1/30			
172/172	_____	212s 1s/step	- loss: 2.4854
Epoch 2/30			
172/172	_____	270s 2s/step	- loss: 1.8253
Epoch 3/30			
172/172	_____	277s 2s/step	- loss: 1.5995
Epoch 4/30			
172/172	_____	304s 2s/step	- loss: 1.4831
Epoch 5/30			
172/172	_____	319s 2s/step	- loss: 1.4122
Epoch 6/30			
172/172	_____	343s 2s/step	- loss: 1.3629
Epoch 7/30			
172/172	_____	309s 2s/step	- loss: 1.3221
Epoch 8/30			
172/172	_____	327s 2s/step	- loss: 1.2876
Epoch 9/30			
172/172	_____	360s 2s/step	- loss: 1.2551
Epoch 10/30			
172/172	_____	294s 2s/step	- loss: 1.2251
Epoch 11/30			
172/172	_____	367s 2s/step	- loss: 1.1955
Epoch 12/30			
172/172	_____	424s 2s/step	- loss: 1.1651
Epoch 13/30			
172/172	_____	423s 2s/step	- loss: 1.1360
Epoch 14/30			
172/172	_____	453s 3s/step	- loss: 1.1049
Epoch 15/30			
172/172	_____	446s 3s/step	- loss: 1.0736
Epoch 16/30			
172/172	_____	360s 2s/step	- loss: 1.0423
Epoch 17/30			
172/172	_____	331s 2s/step	- loss: 1.0119
Epoch 18/30			
172/172	_____	335s 2s/step	- loss: 0.9814
Epoch 19/30			
172/172	_____	530s 3s/step	- loss: 0.9506
Epoch 20/30			
172/172	_____	417s 2s/step	- loss: 0.9209
Epoch 21/30			
172/172	_____	329s 2s/step	- loss: 0.8920
Epoch 22/30			
172/172	_____	408s 2s/step	- loss: 0.8661
Epoch 23/30			
172/172	_____	423s 2s/step	- loss: 0.8411
Epoch 24/30			
172/172	_____	346s 2s/step	- loss: 0.8197
Epoch 25/30			
172/172	_____	337s 2s/step	- loss: 0.7986

```
Epoch 26/30
172/172 _____ 351s 2s/step - loss: 0.7793
Epoch 27/30
172/172 _____ 313s 2s/step - loss: 0.7630
Epoch 28/30
172/172 _____ 339s 2s/step - loss: 0.7488
Epoch 29/30
172/172 _____ 332s 2s/step - loss: 0.7352
Epoch 30/30
172/172 _____ 334s 2s/step - loss: 0.7229
```

Load the Model

We'll rebuild the model from a checkpoint using a `batch_size` of 1 so that we can feed one piece of text to the model and have it make a prediction.

```
model = build_model(VOCAB_SIZE, EMBEDDING_DIM, RNN_UNITS,
                    batch_size=1)
```

Once the model finishes training, we can find the **latest checkpoint** that stores the models weights using the following line.

```
import glob

# FIX: Find the latest .weights.h5 file in the directory manually
list_of_files = glob.glob(os.path.join(checkpoint_dir,
    '*.weights.h5'))
latest_checkpoint = max(list_of_files, key=os.path.getctime)

print("Loading weights from:", latest_checkpoint)

# Load the weights into the model
model.load_weights(latest_checkpoint)
model.build(tf.TensorShape([1, None]))

Loading weights from: ./training_checkpoints\ckpt_30.weights.h5
```

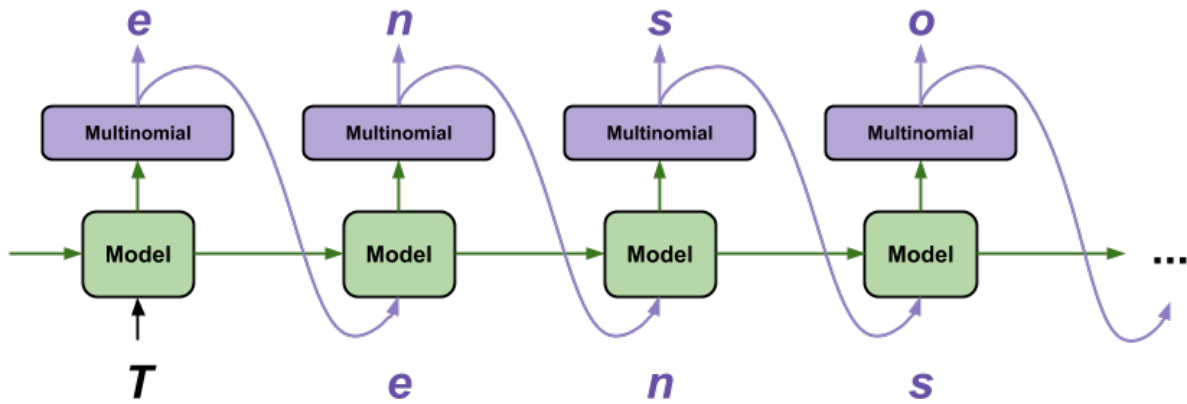
We can load **any checkpoint** we want by specifying the exact file to load.

```
# checkpoint_num = 10
#
model.load_weights(tf.train.load_checkpoint("./training_checkpoints/
ckpt_" + str(checkpoint_num)))
# model.build(tf.TensorShape([1, None]))
```

Generate Text

Generate some text using any starting string we choose.

Generating text with this model is to run it in a loop, and keep track of the model's internal state as you execute it. Each time you call the model you pass in some text and an internal state. The model returns a prediction for the next character and its new state. Pass the prediction and state back in to continue generating text.



```

def generate_text(model, start_string):
    # Evaluation step (generating text using the learned model)

    # Number of characters to generate
    num_generate = 800

    # Converting our start string to numbers (vectorizing)
    input_eval = [char2idx[s] for s in start_string]
    input_eval = tf.expand_dims(input_eval, 0)

    # Empty string to store our results
    text_generated = []

    # Low temperatures results in more predictable text.
    # Higher temperatures results in more surprising text.
    # Experiment to find the best setting.
    temperature = 1.0

    # Here batch size == 1
    model.reset_states()
    for i in range(num_generate):
        predictions = model(input_eval)
        # remove the batch dimension

        predictions = tf.squeeze(predictions, 0)

        # using a categorical distribution to predict the character
        # returned by the model
        predictions = predictions / temperature
        predicted_id = tf.random.categorical(predictions, num_samples=1)

```

```
[-1,0].numpy()

    # We pass the predicted character as the next input to the model
    # along with the previous hidden state
    input_eval = tf.expand_dims([predicted_id], 0)

    text_generated.append(idx2char[predicted_id])

    return (start_string + ''.join(text_generated))

inp = input("Type a starting string: ")
print(generate_text(model, inp))
```

what is happening? marroners: my lord's friends were more advice: yet
do not
sometimes at fine than he is dead less,
Which tender tongues, great manners are request,
To Wets the common people.

SEBASTIAN:
Look worse than e'er I will not dread.
Now, for the flinty rides entreat,
Even in so stopp'd, and wail their looks and if my dust.

KING RICHARD II:
Trust my present villain, not a word more;
I'll go and Richard:' all the world takes;
For I will not read that 'twere to tell than see
Nay, that I saved a joyful bride.

JULIET:
I will bring you so; and 'twere his highness; midits his beard.

AUTOLYCUS:
I have a haste may prove a word to instruct her husband. Verily, I do
learn'd
Shall satisfy too: thou wouldst I give you?
Can it not pout, thou doth talk.

HORTENSIO:
Come, madam, I shall come again with thee s

Sources

1. Chollet François. Deep Learning with Python. Manning Publications Co., 2018.
2. "Text Generation with an RNN : TensorFlow Core." TensorFlow, www.tensorflow.org/tutorials/text/text_generation.
3. "Understanding LSTM Networks." Understanding LSTM Networks -- Colah's Blog, <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>.